Radiografía de un programa y su ecosistema para IA

Programación de Inteligencia Artificial

Housing AI Project — House Price Prediction

California Housing (scikit-learn)

Febrero 2026

Parte A: Anatomía | Parte B: Python vs R | Parte C: JSON & YAML

A

PARTE A · CE (A)

Anatomía de un programa de IA

Proyecto analizado

Housing AI Project — sistema de predicción del precio de viviendas usando el dataset *California Housing*. Implementa un pipeline completo: carga de datos → preprocesado → entrenamiento → evaluación → guardado del modelo, con separación clara entre módulos y configuración externalizada.

1 · Estructura del programa

El proyecto sigue una arquitectura **modular por responsabilidad**, donde cada fichero Python tiene un único propósito bien definido. Esta separación facilita el testing individualizado, la reutilización y el mantenimiento.

```
Proyecto2/
├── config/
│   └── config.json          # Configuración centralizada
├── data/
│   └── new_houses.csv       # Datos de entrada para inferencia
├── outputs/
│   ├── models/              # housing_model.joblib
│   ├── metrics/             # metrics.json
│   └── predictions/         # predictions.csv
├── src/
│   ├── config.py            # Lectura de config.json
│   ├── data.py              # Carga y split del dataset
│   ├── model.py             # Construcción del modelo
│   ├── train.py             # Orquestador principal (entrypoint)
│   ├── evaluate.py          # Cálculo de métricas
│   └── predict.py           # Inferencia sobre nuevos datos
└── requirements.txt         # Dependencias con versiones fijadas
```

`config.py`

Carga el fichero `config.json` y lo deserializa como diccionario Python.

`data.py`

Descarga el dataset California Housing, separa features (`X`) de target (`y`) y

Abstrae la lectura de configuración del resto del código.

realiza la división train/test con parámetros del config.

`model.py`

Construye e instancia el `RandomForestRegressor` de scikit-learn usando los hiperparámetros proporcionados como diccionario.

`evaluate.py`

Calcula métricas de regresión (MAE y RMSE) sobre el conjunto de test. Devuelve un diccionario serializable a JSON.

`train.py`

Punto de entrada principal. Orquesta el pipeline completo y persiste modelo y métricas en disco.

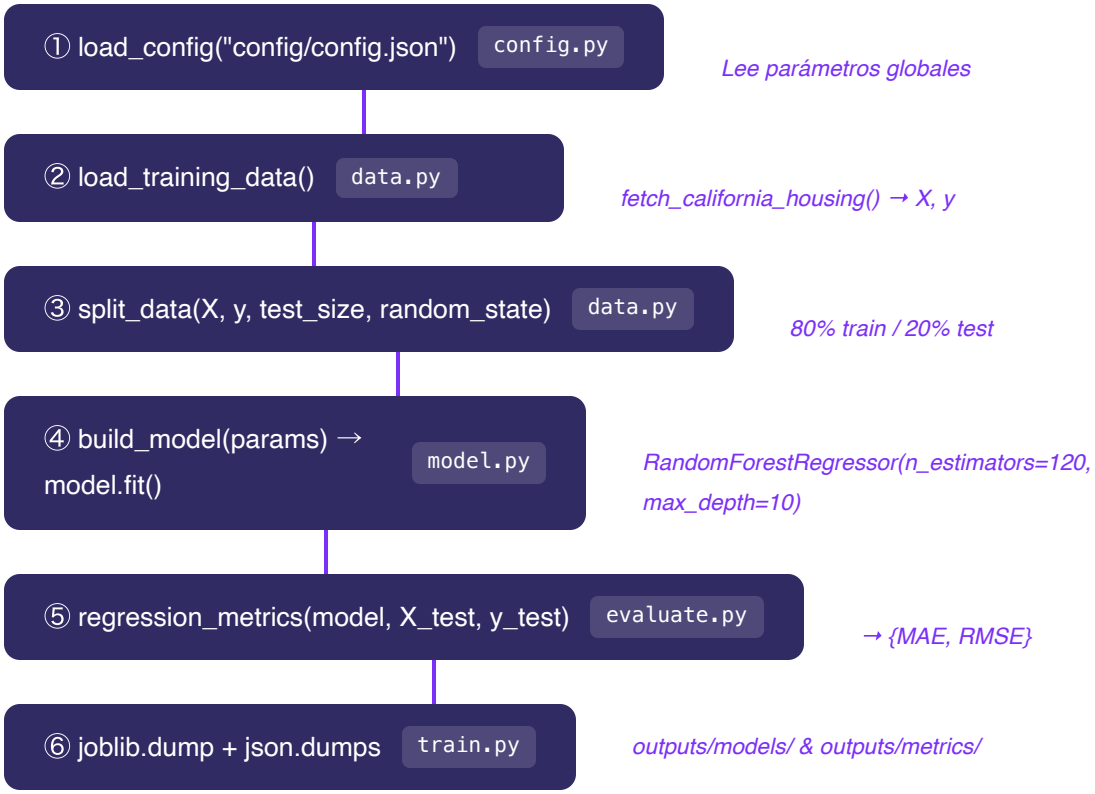
`predict.py`

Carga el modelo serializado y lo aplica sobre un CSV de nuevas viviendas. Guarda predicciones en `outputs/predictions/`.

2 · Flujo de ejecución

El sistema tiene dos *entry points* diferenciados: el pipeline de **entrenamiento** (`train.py`) y el pipeline de **inferencia** (`predict.py`).

Pipeline de entrenamiento: `python -m src.train`



Pipeline de inferencia: `python -m src.predict`

① joblib.load("outputs/models/housing_model.joblib") `predict.py`

② pd.read_csv("data/new_houses.csv") `predict.py`

③ model.predict(new_data) → to_csv() `predict.py`

outputs/predictions/predictions.csv

3 · Responsabilidades de cada bloque

Función	Módulo	Responsabilidad	I/O
`load_config()`	config.py	Deserializa JSON de configuración en un dict Python	path → dict
`load_training_data()`	data.py	Descarga dataset y separa features del target	() → X, y
`split_data()`	data.py	Divide en train/test con semilla de reproducibilidad	X,y,cfg → 4 arrays
`build_model()`	model.py	Instancia el estimador con hiperparámetros del config	params → estimador
`regression_metrics()`	evaluate.py	Calcula y devuelve MAE y RMSE sobre test	model,X,y → dict
`ensure_dir()`	train.py	Crea rutas de salida si no existen (creación idempotente)	path → None
`main()` train	train.py	Orquesta todo el pipeline y persiste modelo + métricas	() → archivos
`main()` predict	predict.py	Carga modelo, predice sobre CSV nuevo y guarda resultado	() → CSV

4 · Puntos críticos

✔ Reproducibilidad — Controlada ✔ Configuración — Externalizada

✔ Dependencias — Fijadas ⚠ Preprocesado — Sin escalado

⚠ Logging — Solo print()

4.1 Reproducibilidad

El parámetro `random_state: 42` se aplica tanto en `train_test_split()` como en `RandomForestRegressor()`, garantizando que el mismo config produce exactamente los mismos resultados en cualquier máquina con la misma versión de scikit-learn. Este es el mecanismo más importante para la reproducibilidad científica en ML.

```
# config.json — control de reproducibilidad
"split": { "test_size": 0.2, "random_state": 42 }
"model": { "params": { "n_estimators": 120, "max_depth": 10 } }
```

4.2 Configuración

Todos los hiperparámetros, proporciones de split y rutas de salida están centralizados en `config/config.json`. El código nunca tiene *hardcoded* valores numéricos significativos. Cambiar el experimento (e.g., aumentar `n_estimators`) no requiere modificar el código Python, solo el JSON.

4.3 Gestión de dependencias

`requirements.txt` fija las versiones exactas de las tres dependencias (`scikit-learn==1.3.0`, `pandas==2.0.3`, `joblib==1.3.2`), asegurando que el entorno es idéntico en todos los equipos. La creación del entorno se reduce a:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

4.4 Lectura y escritura

El sistema gestiona tres tipos de I/O: (1) datos de entrada vía API de sklearn o `pd.read_csv()`, (2) modelo serializado con `joblib.dump/load()` —formato binario eficiente para objetos sklearn—, y (3) métricas como JSON humano-legible. La función `ensure_dir()` utiliza `Path.mkdir(parents=True, exist_ok=True)`, lo que hace la escritura idempotente y tolerante a primeras ejecuciones.

> ⚠ **Área de mejora identificada: preprocesado**

> El pipeline actual no aplica escalado de features. En producción, un `Pipeline` de sklearn con `StandardScaler` garantizaría que el escalado se aplica consistentemente en train e inferencia, eliminando un riesgo habitual de *data leakage*.

PARTE B · CE (B, D)

B Idoneidad de lenguajes para IA: Python vs R

Contexto de la comparativa

Lenguaje elegido para la comparativa: **R**. Ambos lenguajes se evalúan sobre el mismo tipo de tarea: entrenamiento y evaluación de un modelo de regresión sobre un dataset tabular estructurado como California Housing.

Python
scikit-learn · pandas · joblib

vs

R
caret · tidymodels · randomForest

1 · Disponibilidad y madurez de bibliotecas de IA

Criterio	Python	R
Ecosistema ML general	**scikit-learn, XGBoost, LightGBM, CatBoost. Maduros, API unificada, miles de algoritmos.**	caret, tidymodels, mlr3. Competentes para estadística, menos cobertura en DL.
Deep Learning	**TensorFlow, PyTorch, Keras. Ecosistema de referencia.**	**torch (R), keras (R). Wrappers, actualizaciones retrasadas, menor adopción.**
Procesamiento NLP	**Hugging Face Transformers, spaCy, NLTK. Estado del arte.**	**text, tidytext. Adecuado para análisis básico; sin soporte de LLMs nativos.**
Estadística clásica	statsmodels. Funcional pero secundario.	**Base de R. Estadística formal, tests avanzados,**

Criterio	Python	R
		distribuciones. Referencia mundial.

Veredicto: Para tareas de ML/IA aplicada, Python tiene un ecosistema considerablemente más amplio y actualizado. R sobresale en estadística inferencial y análisis exploratorio.

2 · Prototipado vs. industrialización

Dimensión	Python	R
Velocidad de prototipado	★★★★★ Notebooks Jupyter, sintaxis concisa, integración nativa con datos.	★★★★☆ RMarkdown, Quarto excelentes para EDA; tidyverse muy expresivo.
Modularización	**Paquetes, módulos, clases. Refactorización natural. PyPI con >500 000 paquetes.**	Sistema de paquetes CRAN maduro pero menos orientado a software de producción.
Industrialización (MLOps)	**MLflow, DVC, FastAPI, Docker. Tooling de producción nativo.**	**Plumber para APIs. Menos integración con pipelines CI/CD y contenedores.**
Testing	**pytest, unittest. Cultura de testing establecida.**	testthat. Funcional para paquetes estadísticos, menos común en proyectos ML.

3 · Rendimiento

Aspecto	Python	R
Computación numérica	NumPy/SciPy (C/Fortran bajo el capó). Operaciones vectorizadas comparables a R.	Vectorización nativa muy optimizada. Para operaciones estadísticas vectoriales, R es frecuentemente más rápido que Python puro.
GPU/CUDA	**Soporte nativo en PyTorch/TensorFlow/JAX.**	**torch (R) permite GPU, pero el tooling es más limitado y**

Aspecto	Python	R
	Acceso directo a CUDA.	**menos documentado.**
Datos de gran escala	**Dask, Polars, Spark (PySpark). Escalado horizontal nativo.**	data.table (extremadamente rápido en memoria), sparklyr. Limitado fuera de RAM.
Overhead del intérprete	GIL limita paralelismo nativo; compensado con multiprocessing/numba.	Un solo hilo por defecto; parallel (R) para paralelismo básico.

4 · Soporte, comunidad y documentación

Python: comunidad de más de **8 millones de desarrolladores activos** (Stack Overflow 2023). Documentación oficial de scikit-learn considerada un estándar de la industria. Actualizaciones frecuentes y respuesta rápida ante bugs. Bibliografía técnica masiva (O'Reilly, Manning, Arxiv).

R: comunidad más pequeña pero muy especializada en estadística, bioestadística, econometría y ciencias sociales. CRAN tiene >20 000 paquetes revisados. Stack Overflow y RStudio Community son los foros de referencia. Excelente documentación académica; menos recursos orientados a producción de software.

5 · Integración con servicios y plataformas

Plataforma	Python	R
Cloud ML (AWS, GCP, Azure)	**SDK nativos, SageMaker, Vertex AI, Azure ML. Soporte de primera clase.**	Soporte disponible pero indirecto; frecuentemente via contenedores Docker.
APIs REST	**FastAPI, Flask, Django. Despliegue de modelos como servicio en minutos.**	Plumber. Funcional pero con menos herramientas de producción.
Contenedores	**Imágenes Docker oficiales; integración CI/CD fluida.**	Posible con rocker/r-base; mayor complejidad de configuración.

Plataforma	Python	R
Dashboards/reporting	Streamlit, Dash. Buenos pero secundarios.	**Shiny es una referencia para reporting estadístico interactivo.**

6 · Coste de aprendizaje y mantenibilidad

Criterio	Python	R
Curva de aprendizaje inicial	**Sintaxis general. Perfil de desarrollador amplio.**	Paradigma funcional orientado a datos; requiere cambio de mentalidad para desarrolladores.
Mantenibilidad en equipo	**Convenciones (PEP8), linters (flake8, black). Fácil onboarding.**	Estilo heterogéneo. La dualidad base-R vs. tidyverse puede generar confusión.
Perfiles que lo dominan	Data scientists, ML engineers, backend developers.	Estadísticos, investigadores, bioestadísticos, econometras.

Conclusión: ¿Cuándo elegir cada uno?

Python es la opción óptima cuando el objetivo es construir un sistema de IA que va a producción: pipeline de datos reproducible, modelo desplegado como API, integración MLOps, y trabajo en equipo multidisciplinar. Es el lenguaje dominante en la industria y academia de IA/ML.

R es preferible cuando el objetivo es investigación estadística rigurosa, publicación académica con resultados reproducibles, análisis exploratorio profundo o reporting interactivo para audiencias no técnicas. En tareas de regresión y clasificación clásicas, los resultados de ambos son equivalentes; la diferencia está en el ecosistema alrededor.

7 · Ejemplo de código equivalente

La misma tarea (entrenamiento de Random Forest) expresada en ambos lenguajes:

Python (este proyecto)

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestRegressor(n_estimators=120, max_depth=10)
model.fit(X_train, y_train)
joblib.dump(model, "outputs/models/housing_model.joblib")
```

R equivalente

```r
library(tidymodels); library(ranger)

split <- initial_split(data, prop = 0.8, strata = MedHouseVal)
rf_spec <- rand_forest(trees = 120, max_depth = 10) |>
  set_engine("ranger") |> set_mode("regression")
rf_fit <- rf_spec |> fit(MedHouseVal ~ ., data = training(split))
saveRDS(rf_fit, "outputs/models/housing_model.rds")
```

Ambos son igualmente legibles para el dominio. La diferencia principal aparece en las fases de despliegue y mantenimiento en equipo.

C

PARTE C · CE (F)

Lenguajes de marcado: JSON y YAML

En un flujo de IA, los lenguajes de marcado cumplen tres funciones clave: **configuración de experimentos**, **trazabilidad de resultados** e **intercambio de datos entre servicios**. A continuación se muestran los dos formatos utilizados en este proyecto.

1 · JSON — Configuración del experimento

config/config.json — Archivo real del proyecto

```
{
  "split": {
    "test_size": 0.2,          // 20% de datos para evaluación
    "random_state": 42        // semilla de reproducibilidad
  },
  "model": {
    "type": "random_forest",     // identificador del algoritmo
    "params": {
      "n_estimators": 120,      // número de árboles del bosque
      "max_depth": 10           // profundidad máxima de cada árbol
    }
  },
  "output": {
    "model_dir": "outputs/models",
    "metrics_dir": "outputs/metrics",
    "predictions_dir": "outputs/predictions"
  }
}
```

Estructura y semántica

Clave	Tipo	Significado
`split.test_size`	float	Fracción del dataset reservada para test (0.2 = 20%)
`split.random_state`	int	Semilla del generador aleatorio; garantiza reproducibilidad
`model.type`	string	Etiqueta del algoritmo; permite selección condicional en código
`model.params`	object	Hiperparámetros pasados directamente al constructor vía `**params`
`output.*_dir`	string	Rutas relativas para persistir modelo, métricas y predicciones

Justificación del uso de JSON en IA

- **Nativo en Python:** el módulo `json` es parte de la librería estándar; no requiere dependencias adicionales.
- **Légible por máquinas y humanos:** puede ser generado por scripts de búsqueda de hiperparámetros (*grid search*) y también modificado manualmente.
- **Interoperable:** formato universal compatible con REST APIs, bases de datos NoSQL (MongoDB), y plataformas cloud (AWS S3, GCP Storage).
- **Trazabilidad:** el archivo `metrics.json` generado al final del entrenamiento almacena MAE y RMSE, permitiendo comparar experimentos a lo largo del tiempo junto al `config.json` que los produjo.

2 · YAML — Configuración ampliada del experimento

config/config.yaml — Representación equivalente en YAML

```yaml
# Configuración del experimento de entrenamiento
# Housing Price Prediction — California Housing Dataset

experiment:
  name: housing_rf_v1           # identificador del experimento
  dataset: california_housing   # fuente de datos
  version: "1.0.0"
```

```yaml
split:
  test_size: 0.2              # proporción de test
  random_state: 42           # semilla de aleatoriedad

model:
  type: random_forest
  params:
    n_estimators: 120         # número de árboles
    max_depth: 10             # profundidad máxima
    random_state: 42          # reproducibilidad del modelo

output:
  model_dir: outputs/models
  metrics_dir: outputs/metrics
  predictions_dir: outputs/predictions

logging:
  level: INFO
  save_logs: true
  log_path: outputs/logs/train.log
```

Estructura y semántica

Bloque	Significado
`experiment`	Metadatos de trazabilidad: nombre del experimento, dataset y versión del pipeline
`split`	Parámetros del *particionado* del dataset; idénticos al JSON pero más legibles inline
`model.params`	Hiperparámetros del estimador; YAML permite añadir comentarios explicativos por línea sin romper el formato
`logging`	Bloque adicional con nivel de log y ruta de salida; ausente en JSON por concisión

Diferencias clave entre JSON y YAML en contexto IA

Característica	JSON	YAML
Comentarios	**No soportados**	**Sí (`#`). Crítico para documentar hiperparámetros**
Legibilidad humana	Buena	**Excelente. Sin llaves ni comillas obligatorias**
Velocidad de parsing	**Muy rápida**	Ligeramente más lenta (parser más complejo)
Interoperabilidad API	**Estándar universal**	Requiere conversión para REST APIs
Uso típico en IA	Métricas, resultados, intercambio entre servicios	Config de entrenamiento, pipelines (MLflow, DVC, GitHub Actions)

Justificación del uso de YAML en IA

- **Estándar de MLOps:** herramientas como `MLflow`, `DVC`, `Hydra` y `GitHub Actions` utilizan YAML de forma nativa para definir pipelines y workflows.
- **Comentarios inline:** en un fichero de configuración de entrenamiento es crítico poder documentar por qué se eligió un `max_depth=10` o un `n_estimators=120`; JSON no lo permite.
- **Jerarquía legible:** la indentación sustituye a llaves y corchetes, haciendo el fichero más limpio cuando hay múltiples niveles de configuración.
- **Gestión de experimentos:** al combinar `config.yaml` versionado en Git con el modelo guardado, cada commit representa un experimento completamente reproducible.

> **Regla práctica: ¿cuándo usar cada formato?**
>
> Usar **JSON** para resultados y métricas (output de programas, APIs REST, bases de datos). Usar **YAML** para configuración de experimentos y pipelines (archivos editados por humanos, versionados en Git, con necesidad de comentarios explicativos).

SÍNTESIS FINAL

Conclusiones y valoración crítica

6 módulos	100%	3 deps
Arquitectura modular clara	Reproducible con random_state=42	Dependencias fijadas exactamente

Fortalezas del proyecto analizado

- **Separación clara de responsabilidades:** cada módulo tiene una única razón para cambiar (principio SRP).
- **Configuración externalizada:** el `config.json` permite variar experimentos sin tocar el código.
- **Reproducibilidad garantizada:** uso consistente de `random_state` en todas las operaciones aleatorias.
- **Gestión robusta de I/O:** `ensure_dir()` previene errores en primera ejecución; `joblib` serializa modelos sklearn con eficiencia.
- **Dos pipelines diferenciados:** entrenamiento e inferencia están separados, lo que facilita el despliegue independiente de cada uno.

Áreas de mejora identificadas

- **Preprocesado de features:** integrar `sklearn.pipeline.Pipeline` con `StandardScaler` para evitar data leakage entre train y test.
- **Sistema de logging:** sustituir `print()` por el módulo `logging` para niveles (INFO/DEBUG/ERROR) y salida a fichero.
- **Ausencia de tests:** añadir tests unitarios con `pytest` para cada módulo, especialmente para las funciones de I/O.
- **Versionado de experimentos:** integrar `MLflow` o `DVC` para trazar automáticamente métricas, parámetros y artefactos por ejecución.

Python como lenguaje de elección

La elección de Python para este proyecto está plenamente justificada: scikit-learn proporciona una API unificada, consistente y extensamente documentada; pandas facilita la manipulación tabular; y joblib permite serializar objetos sklearn preservando todos sus atributos. El ecosistema de herramientas circundante (MLflow, Docker, FastAPI) facilita la evolución del proyecto desde prototipo hasta sistema productivo sin cambiar de lenguaje ni de paradigma.